



FileMint

Zero-Dependency File Consolidation & Archiving Engine

FileMint

File Consolidation & Archiving Engine

*Comprehensive Technical Synopsis, Architecture Specifications,
Command Reference & Windows Integration Verification Audit*



Application Name	FileMint Desktop Engine
Engine Version	v15.0 (Build 15.0.2 Matrix Tag v15.0.2)
Release Classification	Multi-Platform Enterprise Build (GNU/Linux GNOME & Windows 10/11)
Core Stack & Libraries	Python 3 Standard Library (os, sys, shutil, re, pathlib, json, tkinter), Windows Batch, Shell Scripts
Deployment Distribution	Standalone Executable (ELF / PE .exe), Portable ZIP, Automated Installer Suite
Developer / Engineering	Hari Krishnan
Document Release & Date	System Synopsis v2.0 September 2026

FileMint Privacy Policy

SOFTWARE LICENSE & PRIVACY DECLARATION

Application Name : FileMint Desktop Engine
Version : v15.0 (Build 15.0.10 | Release v15.0.2)
License Type : MIT Open Source License
Copyright : (c) 2026 FileMint Development Team
Developer Website: <https://github.com/encloudme>
Feedback/Issues : <https://github.com/encloudme/FileMint/issues>
Developer Contact: filemint.help@proton.me

TERMS OF USE & PERMISSIONS

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sub-license, and/or sell copies of the Software, subject to the condition that the above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

PRIVACY & OFFLINE ARCHITECTURE GUARANTEE

- 100% Offline Execution: FileMint is built strictly with Python 3 standard library modules (os, sys, shutil, re, pathlib, json, tkinter). It makes zero network, HTTP, or remote API requests.
- Zero Telemetry: No usage tracking, telemetry, error logs, or source code data are ever collected or transmitted.
- Local Workspace Privacy: File scanning, regex validation, file merging, and archiving occur exclusively on your local machine.

DISCLAIMER OF WARRANTY

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NON-INFRINGEMENT.

IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

Documentation Outline and Index:		
Section	Title	Core Focus & Coverage
Section 1	Executive Overview & System Scope	Zero-dependency Python desktop engine, scope, and objectives
Section 2	System Architecture & Repository Structure	Modular repository layout and component mapping
Section 3	Strict Regex Naming Schema & Validation	STRICT_PATTERN regex rules, mandatory prefix and numeric limits
Section 4	Real-Time Versioning & Overwrite Protection	Version sniffing, auto-incrementing, and downward version blocking
Section 5	Formatting Assistant & Collision Avoidance	Interactive Version History Dialog, collision repair, and batch renames
Section 6	Consolidation Execution & Pristine Archiving	File compilation, boundary headers, and shutil archiving sweep
Section 7	GNOME Desktop Integration & Window Focus	XDG hicolor icons, .desktop launcher, and window focus management
Section 8	Command Reference & Administration Toolkit	Complete CLI reference, installer triggers, and environment setups
Section 9	Verification Matrix & 7-Phase Test Summary	7-Phase functional verification benchmarks and 100% pass status
Section 10	Know Issues and Future Enhancement Scope	Know Issues and future enhancement Scope
Section 11	Annexure: Windows Installation & App Testing	23-Step Windows 10/11 screenshot audit & operational verification

1. Executive Overview & System Scope

FileMint is a standalone, zero-dependency Python desktop engine designed to automate combining distributed code base files into unified baseline documents with strict regex schema validation, automated versioning, and pristine archiving. Modern software engineering, technical documentation, and LLM context ingestion frequently require aggregating scattered source files across nested directory trees into a single, structured reference file. Manual copy-pasting is prone to human error, file omissions, broken version histories, and accidental overwrites.

FileMint eliminates these risks through an automated desktop application built strictly with Python 3 standard library modules (os, sys, shutil, re, pathlib, json, tkinter). Operating without third-party pip dependencies, FileMint delivers maximum execution speed, zero supply-chain risk, and instant portability across GNU/Linux GNOME and Microsoft Windows desktop environments.

The engine scans target directories recursively, discovers extension types, filters excluded files or sub-folders, validates baseline output naming rules against a strict regular expression schema, auto-increments version tags, and safely moves legacy baseline builds into an isolated archive directory. Attached below is the screenshot from the app main window, displaying active source folders, extension filters, baseline path selection, etc.

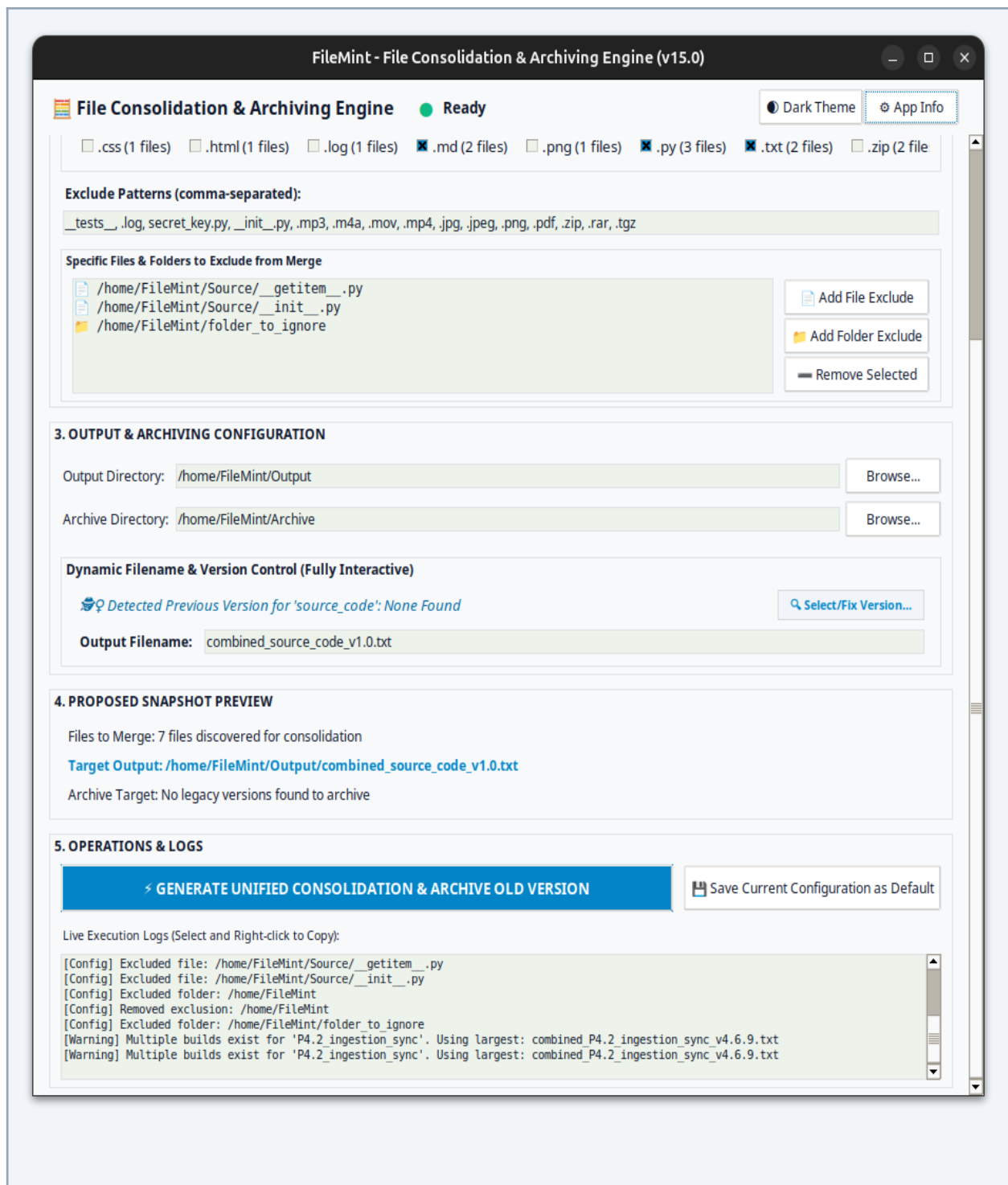


Figure 1: FileMint UI, light-themed main window with active source folders, extension filters, and baseline path selection.

2. System Architecture & Modular Directory Structure

FileMint employs a modular repository layout that decouples core business logic, user interface presentation, configuration management, automated deployment scripts, brand graphics, and test suites.

To ensure long-term maintainability, professional distribution, and seamless test automation, FileMint organizes all assets into dedicated top-level directories:

1. **src/**: Contains the execution scripts viz., **core.py** (business engine, regex validator, file merger, and archiving engine), **gui.py** (Tkinter UI layer, Obsidian dark/light themes, and modal dialogs), and **filemint.py** (primary entry point boot-strapper)
2. **config/**: Stores runtime settings (*appConfig.json*, *fileOpsConfig.json*) and desktop templates (*filemint.desktop*).
3. **installers/**: Houses cross-platform automated deployment scripts — *install.sh* and *uninstall.sh* for Linux GNOME, *install_windows.bat* and *uninstall_windows.bat* for Windows 10/11.
4. **assets/**: Holds high-resolution application branding PNG graphics (*filemint.png*, *icon.png*), GitHub banners etc.
5. **archive/**: Contains packaged repository release zips and legacy assets.
6. **documentation/**: Stores technical guides, requirements (*ProjectScope.md*), Quick Start Guide, and system synopsis documents.
7. **tests/**: Contains verification logs and the comprehensive 7-Phase Test Summary Report.
8. **Output/Archive Workspaces**: Standard user workspace folders (e. g., *~/Documents/FileMint/Output* and *~/Documents/FileMint/Archive*) where baseline builds and archived compilations reside.

Directory	Key Components	System Role & Responsibilities
src/	core.py, gui.py, filemint.py	Core application execution scripts, UI rendering, and entry point.
config/	appConfig.json, filemint.desktop, fileOpsConfig.json	Runtime settings, path macros, and GNOME launcher templates.
installers/	install.sh, install_windows.bat, uninstall.sh	Automated platform deployment and desktop environment integration.
assets/ assets/icons	filemint.png, icon.png	XDG hicolor theme icons and application window brand graphics.
archive/	Archive files	Repository archives and baseline release backups.
documentation/	Quick_Start_Guide.md/ Quick_Start_Guide_FileMint.md, README.md	User manuals, operational guides, and architectural specifications.
tests/	Test_Summary_Report.md	Comprehensive verification logs and 7-phase test suite results.

```

$ cd FileMint/
$ tree -I '.__pycache__|combined*'
.
├── archive
├── assets
│   ├── filemint_github_banner.png
│   └── icons
│       ├── edited_icon.jpeg
│       ├── filemint.png
│       ├── icon.png
│       ├── icon_with_reflection.jpeg
│       └── legacy_icon.png
├── config
│   ├── appConfig.json
│   ├── filemint.desktop
│   ├── fileOpsConfig.json
│   └── fileOpsConfig_prev.json
├── documentation
│   ├── DEV_NOTES.md
│   ├── FileMint_Project_Synopsis.docx
│   └── ProjectScope.md
├── installers
│   ├── install.sh
│   ├── install_windows.bat
│   ├── Quick_Start_Guide.md
│   └── uninstall.sh
├── README.md
├── src
│   ├── core.py
│   ├── filemint.py
│   └── gui.py
└── tests
    └── Test_Summary_Report.md

```

Figure 2: The FileMint app code repository structure
(developer mode, current structure may vary)

3. Strict Regex Naming Schema & Validation Rules

FileMint strictly enforces a baseline output filename schema using a compiled regular expression that mandates approved prefixes and between 2 and 4 numeric version segments.

To maintain seamless version tracking and prevent file collisions across automated workflows, baseline compilation files in the Output Directory must strictly conform to STRICT_PATTERN in core.py:

Pattern: `^(combined|merged|merge)_([\s]?[vV](\d+(?:\.\d+){1,3})\.txt$`

Schema Specifications:

- **Mandatory Prefix:** Filename must begin with 'combined_', 'merged_', or 'merge_'.
- **Core Project Name:** A non-empty descriptive core identifier (e.g., 'app', 'filemint'), without whitespace or tabs.
- **Version Identifier:** Must start with 'v' or 'V' followed by 2 to 4 numeric segments separated by dots (e.g., v1.0, v1.2.3, or v1.2.3.4).
- **Single-segment versions** (e.g., v1) and versions with 5 or more segments (e.g., v1.0.2.9.5) are strictly rejected as non-compliant.
- **File Names:** Must not contain whitespace or tabs. (can use '_' or '-' as separator)

Filename Candidate	Validation Result	Technical Compliance Reason
combined_app_v1.0.txt	VALID	Prefix 'combined_', valid core, 2 numeric segments (1.0).
merged_core_v1.2.3.txt	VALID	Prefix 'merged_', valid core, 3 numeric segments (1.2.3).
merge_engine_v4.3.2.1.txt	VALID	Prefix 'merge_', valid core, 4 numeric segments (4.3.2.1).
output_app_v1.0.txt	INVALID	Rejected: Unapproved prefix 'output_' (must be combined/merged/merge).
combined_app_v1.txt	INVALID	Rejected: Only 1 numeric segment (minimum 2 required).
merged_app_v1.0.2.9.5.txt	INVALID	Rejected: 5 numeric segments (maximum 4 allowed).
merged_source code_v1.2.txt	INVALID	Rejected: whitespace present in file name.

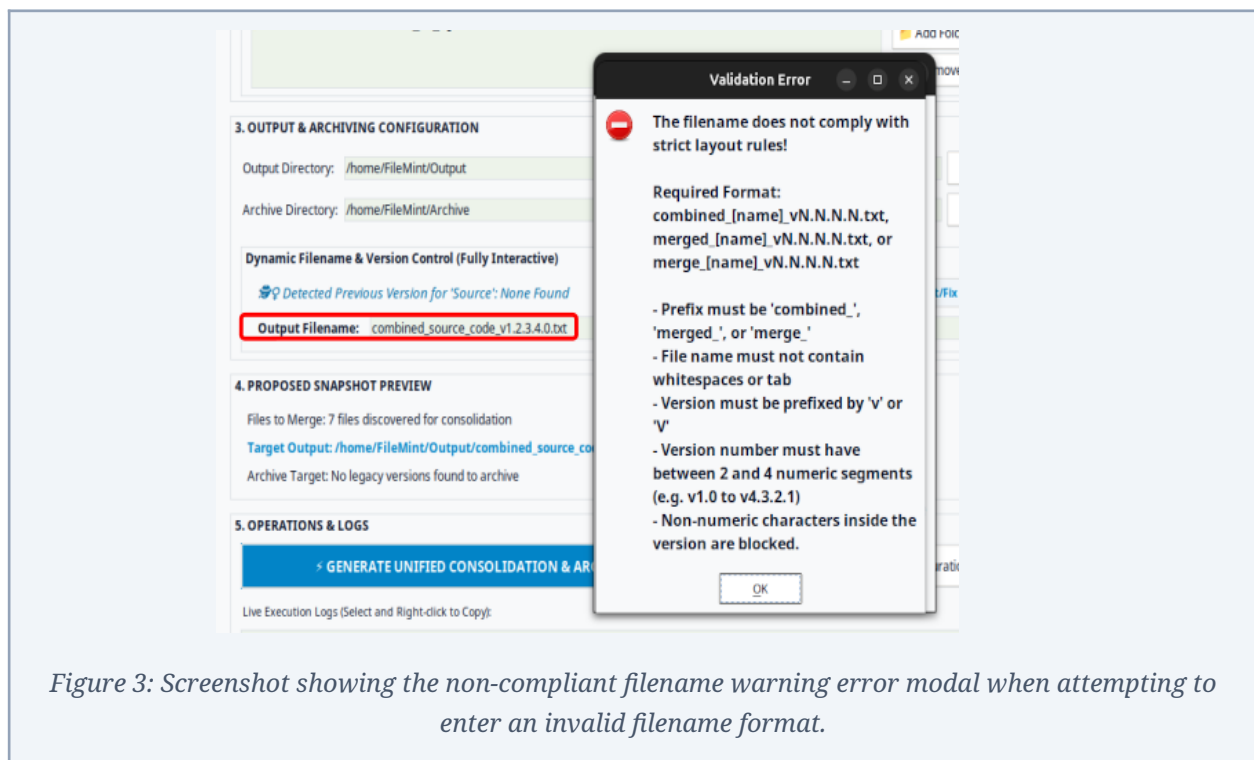


Figure 3: Screenshot showing the non-compliant filename warning error modal when attempting to enter an invalid filename format.

4. Real-Time Versioning & Overwrite Protection Workflow

FileMint automatically scans the target Output Directory to recommend the next incremental version, block downward version regressions, and require explicit confirmation before overwriting existing builds.

To ensure continuous build history without accidental overwrites, FileMint executes a three-tier protection sequence in `gui.py`:

1. Automatic Version Sniffing & Auto-Increment: Upon selecting an Output Directory, `sniff_previous_version()` scans for existing compliant files matching the core project name. It parses version numbers into numerical tuples, identifies the highest existing version on disk, and auto-populates the UI input box with the next sequential build number (e.g., if `combined_app_v1.0.0.0.txt` exists, it pre-populates `combined_app_v1.0.0.1.txt`).
2. Downward Version Blocking: If a user manually edits the input field to enter a version tuple lower than the existing maximum build (e.g., entering `v0.9` when `v1.0` exists), the engine immediately flags a version regression error and halts execution.
3. Overwrite Protection Modal: If the user explicitly enters an exact version matching an existing file on disk, FileMint triggers merge confirmation dialog, requiring explicit user confirmation before replacing the file.

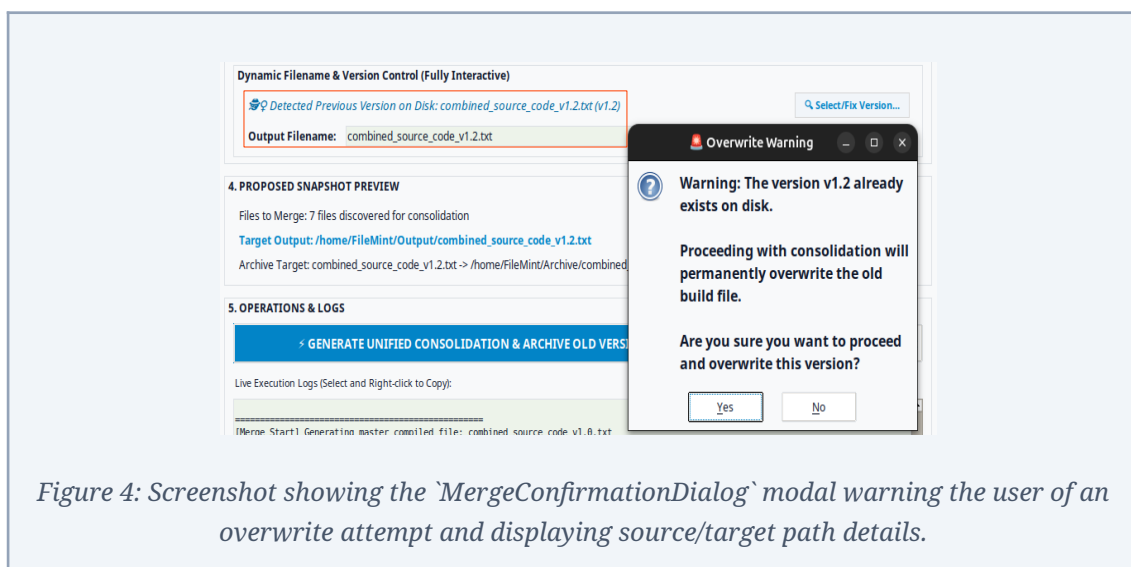


Figure 4: Screenshot showing the 'MergeConfirmationDialog' modal warning the user of an overwrite attempt and displaying source/target path details.

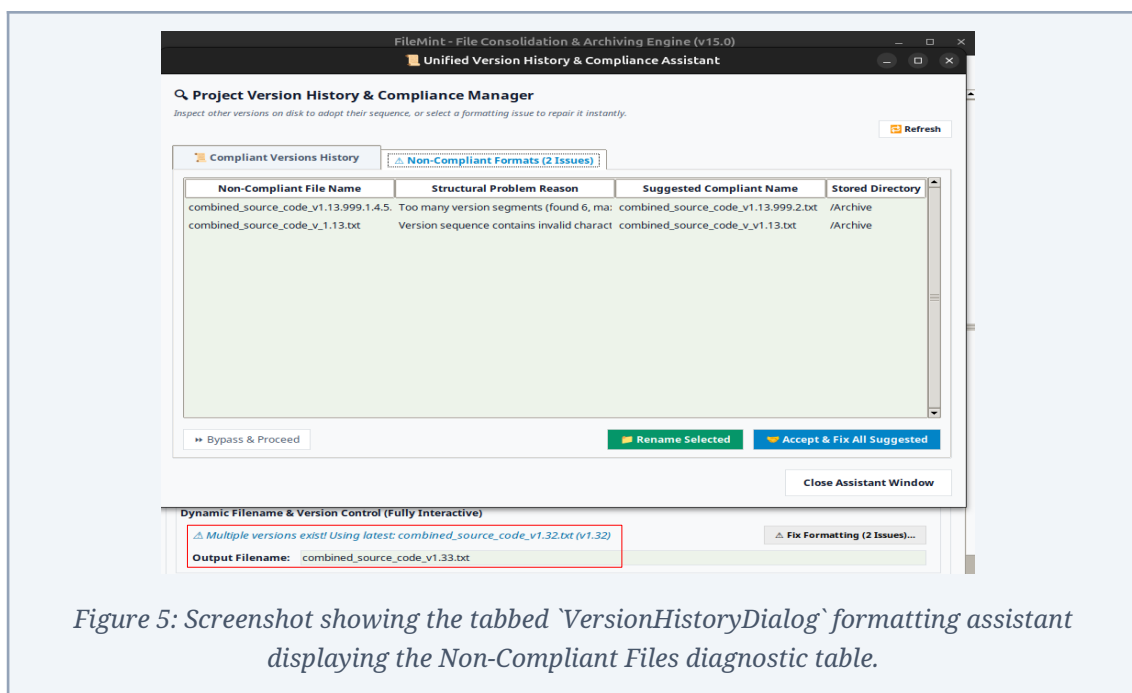
5. Formatting Assistant & Collision Avoidance Logic

The interactive VersionHistoryDialog assistant audits non-compliant legacy files, automatically calculates collision-free repair names, and batch-renames files on disk in a single pass.

Legacy repositories or imported output directories frequently contain malformed baseline files (e.g., missing prefixes, 1 segment, or 5+ segments). FileMint provides an interactive compliance repair assistant accessed via the '⚠️ Fix Formatting' button.

Diagnostic & Repair Workflow:

- **Directory Audit:** VersionHistoryDialog categorizes all files in the target directory into 'Compliant History' vs 'Non-Compliant Files'.
- **Collision-Free Auto-Bumping:** When repairing malformed version strings (e.g., merged_app_v1.0.2.9.5.txt), the engine extracts the base tuple (1.0.2.9). If v1.0.2.9.txt already exists on disk, it automatically increments the last segment until a free slot is found (e.g., v1.0.2.10.txt).
- **Batch Disk Renaming:** Clicking '👉 Accept & Fix All Suggested' renames all non-compliant files on disk in a single atomic pass.



6. Consolidation Execution & Pristine Archiving Engine

During execution, Consolidation Engine writes a unified reference compilation with metadata headers, then sweeps older baseline builds from Output into Archive with collision timestamp guards.

The physical merge and archiving workflow consists of two distinct phases handled by core.py:

Phase A — Unified File Compilation (perform_merge_write):

- Iterates through all discovered, non-excluded candidate files across selected source folders.
- Inserts standardized top-level metadata banners (timestamp, file count, folder paths).
- Places explicit boundary markers (===== FILE: path/to/file =====) before each file's raw content.
- Enforces target self-exclusion to prevent destination compilation files from reading into themselves if placed inside a source folder.

Phase B — Pristine Archiving Sweep (execute_archive_cleanup):

- Scans the Output Directory for older compilation files matching the project core name.
- Relocates legacy baseline builds into the Archive Directory using shutil.move().
- If an archived file with an identical name already exists in Archive, the engine appends a Unix modification timestamp ('_archived_{timestamp}') to ensure zero data loss.

```
# =====  
# 📁 COMPILER ENGINE: FileMint (v15.0.0)  
# 📄 BASELINE COMPILATION: combined_source_code_v1.33.txt  
# =====  
  
### =====  
FILE: app.py PATH: /home/FileMint/Source/app.py  
=====
```

```
import os  
import sys  
import shlex  
import shutil  
import datetime  
  
# Import relevant PyQt6 libraries and modules  
from PyQt6.QtWidgets import (  
    QApplication, QMainWindow, QWidget, QVBoxLayout, QHBoxLayout,  
    QTableWidgetItem, QPushButton, QLabel, QStackedWidget
```

Figure 6: Screenshot showing the generated unified reference file content with metadata headers and file boundary markers.

7. GNOME Desktop Integration & Window Focus Management

FileMint integrates natively into GNOME desktops via XDG hicolor icon themes and desktop entry launchers, while eliminating GTK background window dropping.

Linux GNOME Desktop Integration:

- Installer automation (installers/install.sh) deploys application branding icons across all standard XDG hicolor theme size buckets (16x16, 32x32, 48x48, 64x64, 128x128, 256x256, 512x512) under `~/.local/share/icons/hicolor/`.
- Registers desktop launcher config/filemint.desktop into `~/.local/share/applications/` with dynamic `USER_HOME` expansion.
- Refreshes icon cache via `gtk-update-icon-cache` and updates application database via `update-desktop-database`

Window Manager Focus Management:

In GNOME (Mutter window manager), closing native file dialogs or popups can cause the main window to drop behind other open desktop windows. FileMint resolves this in `gui.py` by enforcing explicit parent binding (`parent=self`) on all modals, followed by `self.lift()` and `self.focus_force()` upon modal dismissal.

8. Development Phase Command Reference & Administration Toolkit

A comprehensive reference of terminal commands used during the development, testing, and system deployment phases of FileMint. Below is a structured catalog of CLI commands and utility shell scripts used during

 Notice:

- **Development Use Only:** The following commands and scripts are for development, testing, and debugging.
- **Production Deployments:** Use official v15.0.2 release archives and platform installers.
- **Documentation:** See *README.md* or *Quick_Start_Guide.md* for complete installation procedures.

1. Repository & Structure Inspection:

```
tree -I '__pycache__|combined*|files_folders_to_test'  
desktop-file-validate ~/.local/share/applications/filemint.desktop
```

2. Desktop Registration & Icon Cache Refresh:

```
chmod +x installers/install.sh installers/uninstall.sh  
./installers/install.sh  
gtk-update-icon-cache -f -t ~/.local/share/icons/hicolor  
update-desktop-database ~/.local/share/applications
```

3. Application Launch Commands:

```
python3 src/filemint.py           # Direct terminal launch  
gtk-launch filemint               # GNOME desktop launcher trigger  
pythonw src/filemint.py           # Windows background launch (no  
console)
```

4. Dependency Audits & System Libraries:

```
python3 -c "import tkinter; print(tkinter.TkVersion)"  
sudo apt install python3-tk          # Debian / Ubuntu / Pop!_OS  
sudo dnf install python3-tkinter     # Fedora / RHEL  
sudo pacman -S tk                   # Arch Linux
```

5. System Reset & Un-installation:

```
rm -f ~/.config/filemint-app/appConfig.json config/appConfig.json  
./installers/uninstall.sh
```

9. Verification Matrix & 7-Phase Test Summary

FileMint has passed a rigorous 7-phase end-to-end verification suite with 100% test pass rate across all functional, GUI, and integration benchmarks. The test suite evaluated FileMint across 7 distinct operational phases to ensure complete stability and compliance:

Phase	Test Area	Status	Key Verification Highlights
Phase 1	Launch & Installer	100% PASS	Verified CLI bootstrapper, desktop shortcut registration, and uninstall scripts.
Phase 2	Regex Naming Engine	100% PASS	Verified STRICT_PATTERN validation rules; blocked non-compliant prefixes and segment bounds.
Phase 3	Versioning & Overwrites	100% PASS	Verified auto-incrementing, downward version blocking, and overwrite confirmation popups.
Phase 4	Formatting Assistant	100% PASS	Verified VersionHistoryDialog audit, collision auto-bumping, and batch disk renames.
Phase 5	Filtering & Exclusions	100% PASS	Verified multi-file exclusion pickers, extension discovery, and target self-exclusion.
Phase 6	Merge & Pristine Archive	100% PASS	Verified master compilation writing, legacy build relocation, and timestamp collision guards.
Phase 7	UI Themes & Persistence	100% PASS	Verified Obsidian dark/light themes, dual JSON config saving, and clean exit clearing.

10. Know Issues & Future Enhancement Scope

1. The dark-styled theme has limitations in rendering colour schemes to the complete extent, especially in pop-up dialog boxes.
2. FileMint app is customized app, it can only add/delete source folders, recursively search for contents. 'File exclusions' option and 'folders omitted' should be used to select required files to merge.
3. The app will only scan source, target and archive folders.
4. For selecting folders, user has to double click the folder icon, and then click "OK" to proceed. Highlighting the folder or single click would not select the folder.

11. Annexure: Windows Installation & App Testing Verification Suite

This annexure documents the end-to-end validation suite executed on Microsoft Windows 10/11 environments. Using a comprehensive set of 23 diagnostic screenshots (00_filemint_win_installation_success.png ⇒ 22_filemint_win_shortcut_remove.png, available in assets/screenshots/windows/).

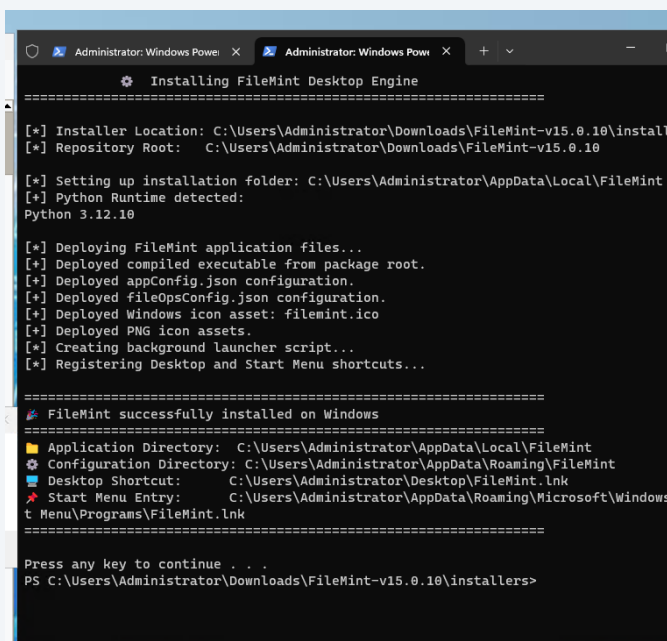
This verification audit traces every operational life-cycle phase:

- Automated installation script execution
- Desktop/start menu shortcut registration
- GUI initialization
- Filter configuration
- Version control auto-incrementing,
- Regex validation error traps
- Formatting assistant batch repair
- JSON configuration persistence and backup creation
- Post-build tree inspection
- Uninstallation verification.

Figure W-00: Windows PowerShell Automated Installer Execution

Source Asset: 00_filemint_win_installation_success.png

Status: 100% VERIFIED



```
Administrator: Windows Power  x  Administrator: Windows Power  x  +  v  -  [
Installing FileMint Desktop Engine
=====
[*] Installer Location: C:\Users\Administrator\Downloads\FileMint-v15.0.10\install
[*] Repository Root: C:\Users\Administrator\Downloads\FileMint-v15.0.10
[*] Setting up installation folder: C:\Users\Administrator\AppData\Local\FileMint
[*] Python Runtime detected:
Python 3.12.10
[*] Deploying FileMint application files...
[*] Deployed compiled executable from package root.
[*] Deployed appConfig.json configuration.
[*] Deployed fileOpsConfig.json configuration.
[*] Deployed Windows icon asset: filemint.ico
[*] Deployed PNG icon assets.
[*] Creating background launcher script...
[*] Registering Desktop and Start Menu shortcuts...
=====
FileMint successfully installed on Windows
=====
Application Directory: C:\Users\Administrator\AppData\Local\FileMint
Configuration Directory: C:\Users\Administrator\AppData\Roaming\FileMint
Desktop Shortcut: C:\Users\Administrator\Desktop\FileMint.lnk
Start Menu Entry: C:\Users\Administrator\AppData\Roaming\Microsoft\Windows
t Menu\Programs\FileMint.lnk
=====
Press any key to continue . . .
PS C:\Users\Administrator\Downloads\FileMint-v15.0.10\installers>
```

System Verification & Test Findings:

Administrator PowerShell executing install-filemint-win.bat inside C:\Users\Administrator\Downloads\FileMint-v15.0.10.

Detects Python 3.12.10 runtime, Deploys application files to AppData\Local\FileMint, creates appConfig.json and fileOpsConfig.json in AppData\Roaming\FileMint, stages filemint.ico, and

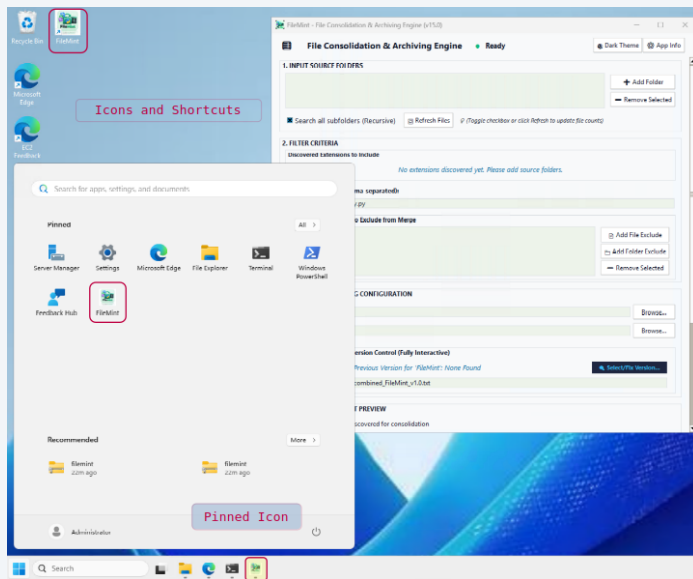
Registers shortcuts on Desktop and Start Menu.

Audit Check: Verified against Windows 10/11 x64 environment with Python 3.12.10 runtime.

Figure W-01: Desktop Shortcuts, Start Menu Pinned Icon & App Launch

Source Asset: 01_filemint_win_desktop_icons_launch.png

Status: 100% VERIFIED



System Verification & Test Findings:

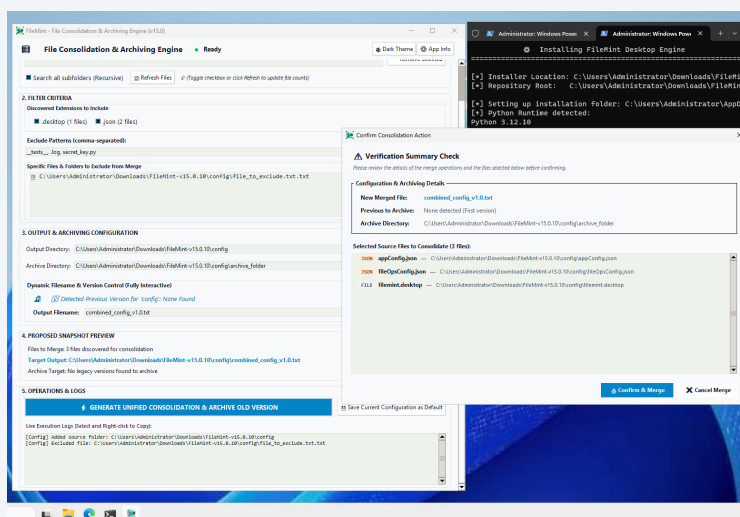
Verifies creation of custom-branded FileMint desktop icon, Start Menu Pinned shortcut, Taskbar launcher, and initial FileMint main window in Light Theme ready state.

Audit Check: Verified against Windows 10/11 x64 environment with Python 3.12.10 runtime.

Figure W-02: Verification Summary Check Modal & Source Folder Selection

Source Asset: 02_filemint_win_merge_consolidation.png

Status: 100% VERIFIED



System Verification & Test Findings:

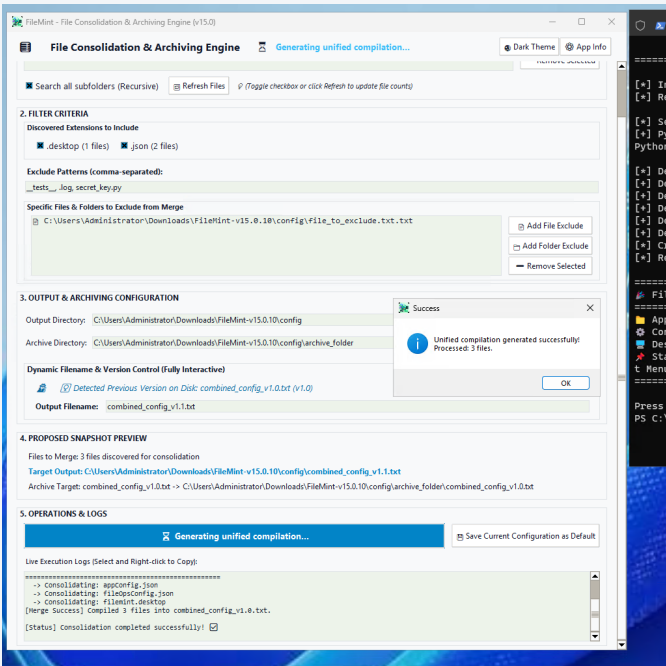
Active source folder C:\Users\Administrator\Downloads\FileMint-v15.0.10\config configured with extension filters (.desktop, .json), exclude patterns (_tests_, .log, secret_key.py), and specific exclude list (file_to_exclude.txt). Verification Summary Check modal confirms 3 target candidate files: appConfig.json, fileOpsConfig.json, and filemint.desktop.

Audit Check: Verified against Windows 10/11 x64 environment with Python 3.12.10 runtime.

Figure W-03: Compilation Execution Success & Live Log Feedback

Source Asset: 03_filemint_win_merge_success.png

Status: 100% VERIFIED



System Verification & Test Findings:

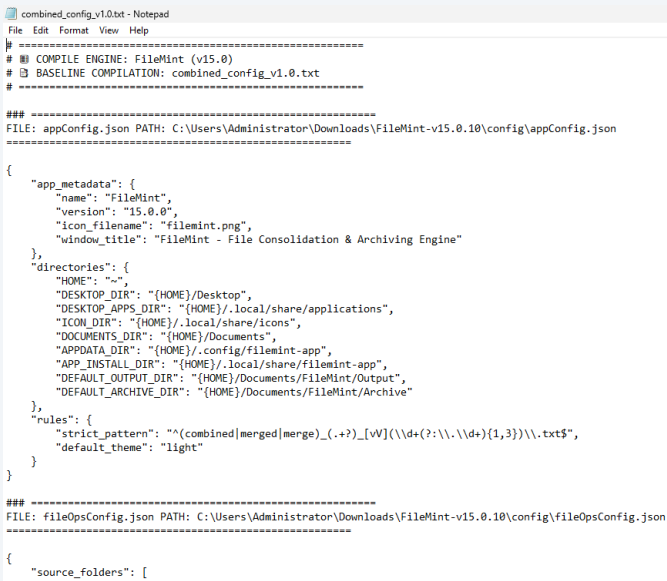
Displays Success modal ('Unified compilation generated successfully! Processed 3 files.'). Version control automatically detected existing combined_config_v1.0.txt (v1.0) on disk and auto-incremented target output to combined_config_v1.1.txt. Live execution logs show step-by-step consolidation stream.

Audit Check: Verified against Windows 10/11 x64 environment with Python 3.12.10 runtime.

Figure W-04: Unified Baseline Output File Inspection in Notepad

Source Asset: 04_filemint_win_merged_file.png

Status: 100% VERIFIED



System Verification & Test Findings:

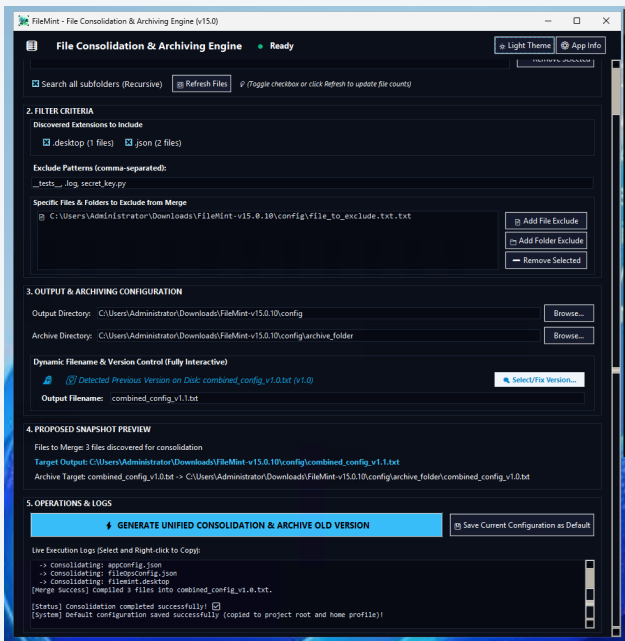
Inspection of generated combined_config_v1.0.txt in Windows Notepad. Displays top-level compilation header (# COMPILE ENGINE: FileMint v15.0), project metadata, and explicit file boundary markers before raw file content.

Audit Check: Verified against Windows 10/11 x64 environment with Python 3.12.10 runtime.

Figure W-05: Obsidian Dark Theme UI Rendering

Source Asset: 05_filemint_win_dark_theme.png

Status: 100% VERIFIED



System Verification & Test Findings:

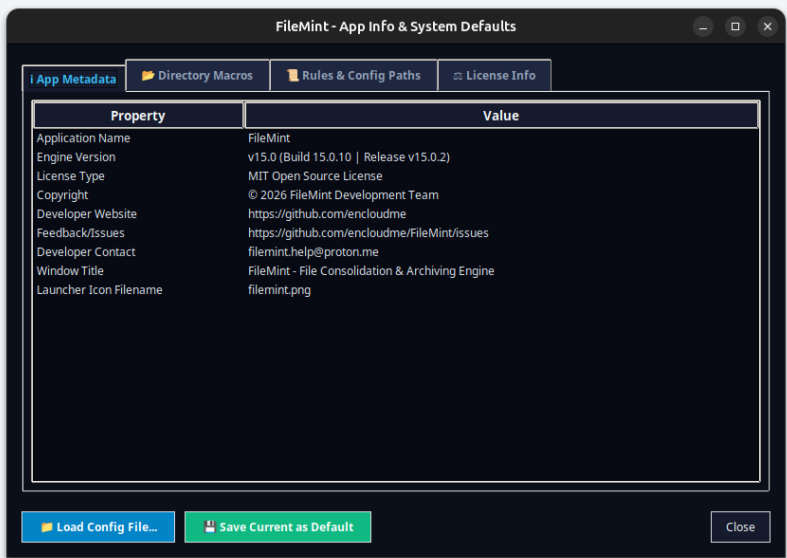
Full UI rendered in high-contrast Obsidian Dark Theme. Validates dark color palette (#1E1E1E background), accent buttons, and active workspace configurations.

Audit Check: Verified against Windows 10/11 x64 environment with Python 3.12.10 runtime.

Figure W-06: App Metadata & System Defaults Modal

Source Asset: 06_filemint_win_app_info.png

Status: 100% VERIFIED



System Verification & Test Findings:

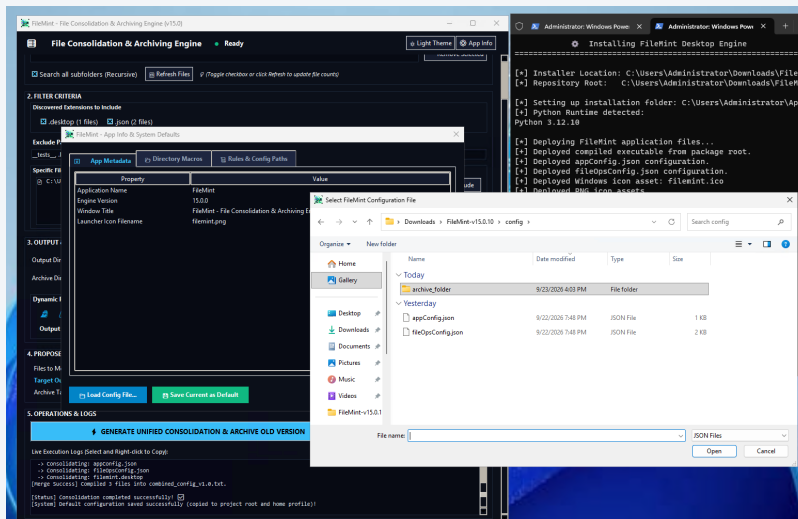
Inspection of 'App Metadata' tab inside App Info & System Defaults dialog. Displays Application Name (FileMint), Engine Version (15.0.0), Window Title, and Icon Filename.

Audit Check: Verified against Windows 10/11 x64 environment with Python 3.12.10 runtime.

Figure W-07: Custom Configuration File Browser Dialog

Source Asset: 07_filemint_win_config_select.png

Status: 100% VERIFIED



System Verification & Test Findings:

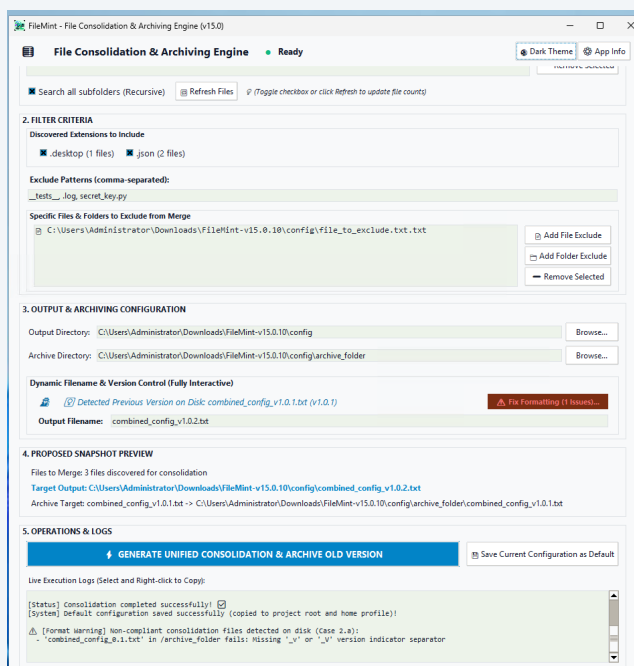
Open File dialog browsing C:\Users\Administrator\Downloads\FileMint-v15.0.10\config to select or import custom appConfig.json or fileOpsConfig.json files.

Audit Check: Verified against Windows 10/11 x64 environment with Python 3.12.10 runtime.

Figure W-08: Non-Compliant Format Warning & 'Fix Formatting' Action Button

Source Asset: 08_filemint_win_fix_formatting.png

Status: 100% VERIFIED



System Verification & Test Findings:

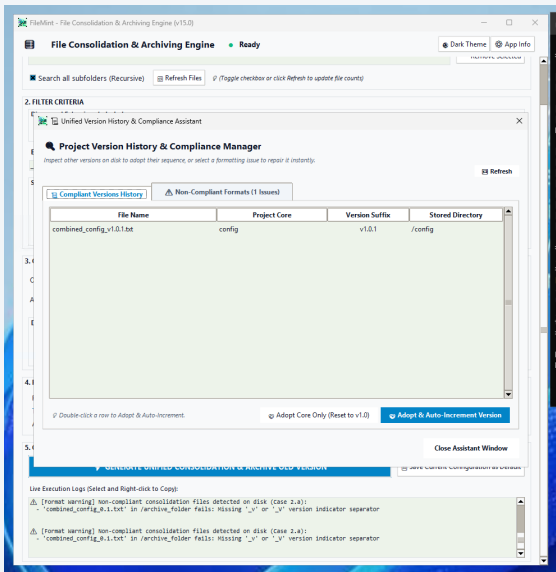
Engine flags a red 'Fix Formatting (1 Issues...)' warning button. Detected non-compliant file combined_config_0.1.txt in archive_folder missing required '_v' version separator.

Audit Check: Verified against Windows 10/11 x64 environment with Python 3.12.10 runtime.

Figure W-09: Unified Version History Assistant — Compliant History Tab

Source Asset: 09_filemint_win_compliant_versions_before.png

Status: 100% VERIFIED



System Verification & Test Findings:

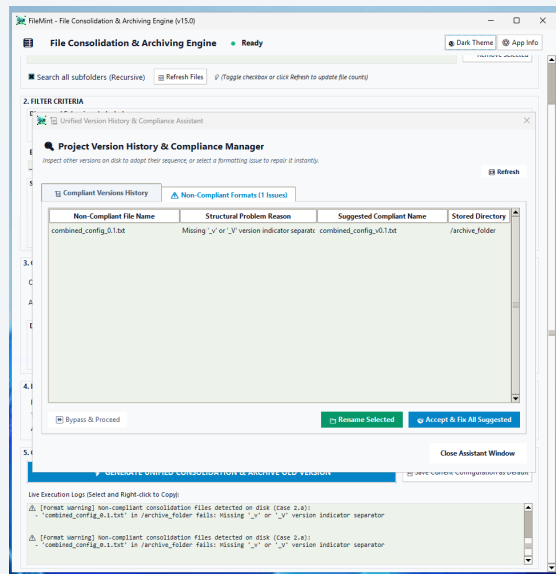
Version History Assistant displaying existing compliant file combined_config_v1.0.1.txt under project core 'config', version suffix v1.0.1, stored in /config.

Audit Check: Verified against Windows 10/11 x64 environment with Python 3.12.10 runtime.

Figure W-10: Unified Version History Assistant — Non-Compliant Formats Tab

Source Asset: 10_filemint_win_non-compliant_format.png

Status: 100% VERIFIED



System Verification & Test Findings:

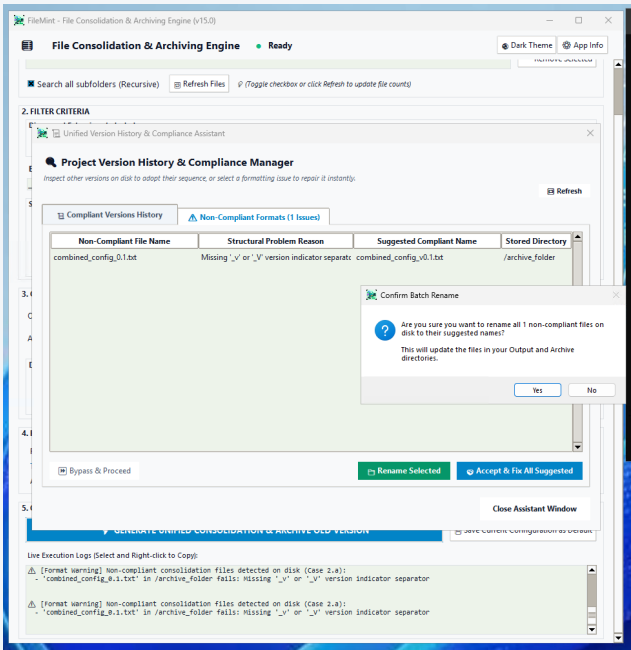
Assistant flags non-compliant file combined_config_0.1.txt, specifies problem reason ('Missing '_v' or '_V' version indicator separator'), and suggests compliant name combined_config_v0.1.txt.

Audit Check: Verified against Windows 10/11 x64 environment with Python 3.12.10 runtime.

Figure W-11: Confirm Batch Rename Confirmation Modal

Source Asset: 11_filemint_win_batch_rename.png

Status: 100% VERIFIED



System Verification & Test Findings:

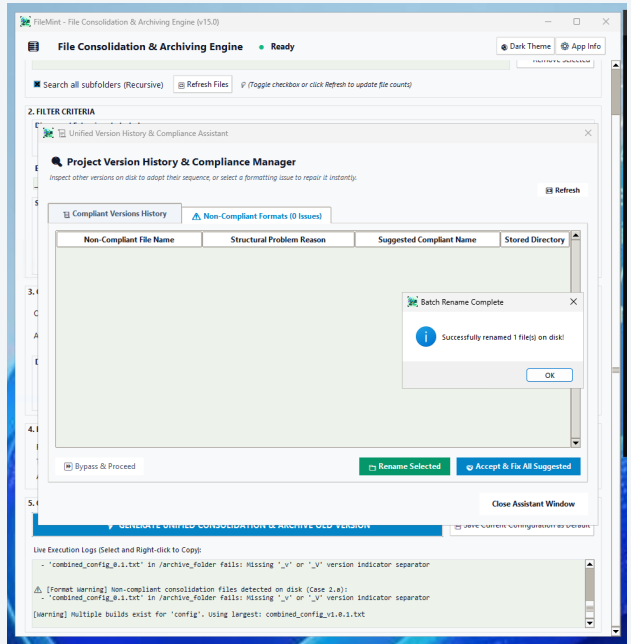
Popup modal confirming batch repair action: 'Are you sure you want to rename all 1 non-compliant files on disk to their suggested names?'

Audit Check: Verified against Windows 10/11 x64 environment with Python 3.12.10 runtime.

Figure W-12: Batch Rename Completion Dialog

Source Asset: 12_filemint_win_rename_success.png

Status: 100% VERIFIED



System Verification & Test Findings:

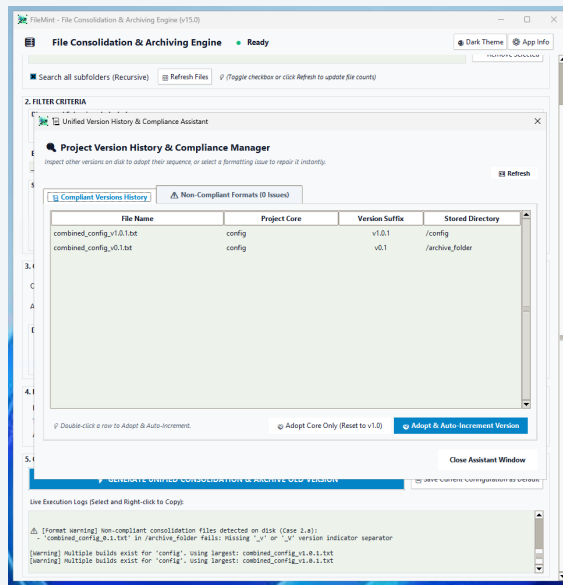
Completion popup confirming atomic disk repair: 'Successfully renamed 1 file(s) on disk!'. Live log displays system status update.

Audit Check: Verified against Windows 10/11 x64 environment with Python 3.12.10 runtime.

Figure W-13: Compliant Versions History Post-Repair State

Source Asset: 13_filemint_win_compliant_version_after.png

Status: 100% VERIFIED



System Verification & Test Findings:

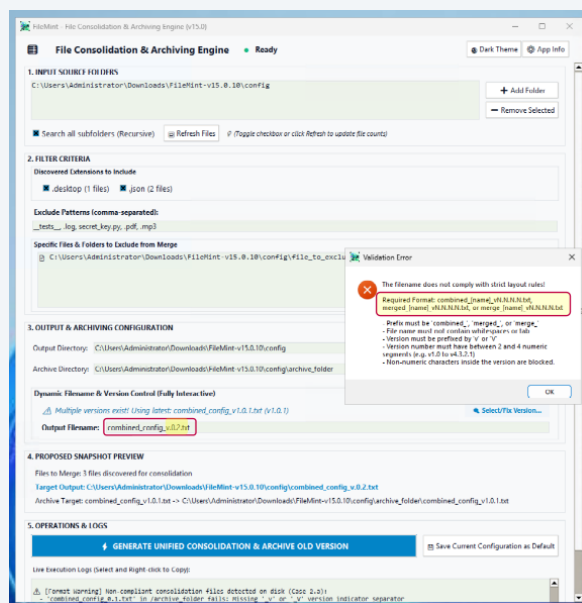
Version History Assistant after repair. Displays both combined_config_v1.0.1.txt and repaired combined_config_v0.1.txt in the compliant history table.

Audit Check: Verified against Windows 10/11 x64 environment with Python 3.12.10 runtime.

Figure W-14: Validation Error Modal — Malformed Pattern Rule Trap

Source Asset: 14_filemint_win_pattern_error.png

Status: 100% VERIFIED



System Verification & Test Findings:

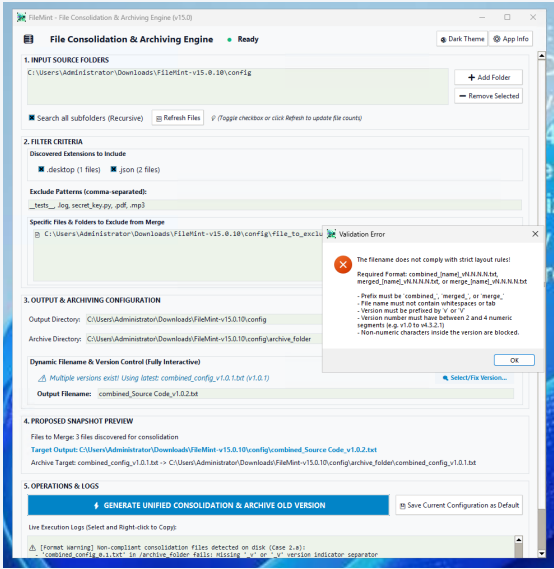
Validation Error dialog blocking output filename combined_config_v0.2.txt. Displays required schema rules: prefix must be combined_, merged_, or merge_, version must be prefixed by 'v' or 'V', version number must have between 2 and 4 numeric segments, non-numeric characters inside the version are blocked.

Audit Check: Verified against Windows 10/11 x64 environment with Python 3.12.10 runtime.

Figure W-15: Validation Error Modal — Core Identifier Space/Special Character Guard

Source Asset: 15_filemint_win_fname_error.png

Status: 100% VERIFIED



System Verification & Test Findings:

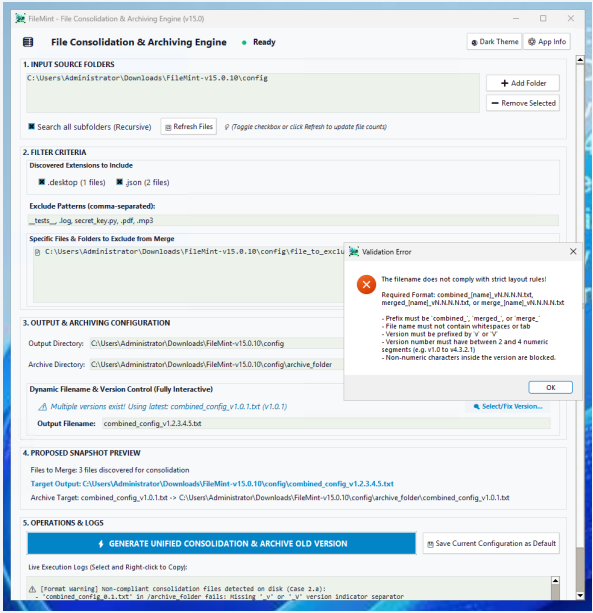
Validation Error dialog blocking filename combined_source Code_v1.0.2.txt due to spaces in the core identifier name.

Audit Check: Verified against Windows 10/11 x64 environment with Python 3.12.10 runtime.

Figure W-16: Validation Error Modal — Segment Count Overflow Trap

Source Asset: 16_filemint_win_version_error.png

Status: 100% VERIFIED



System Verification & Test Findings:

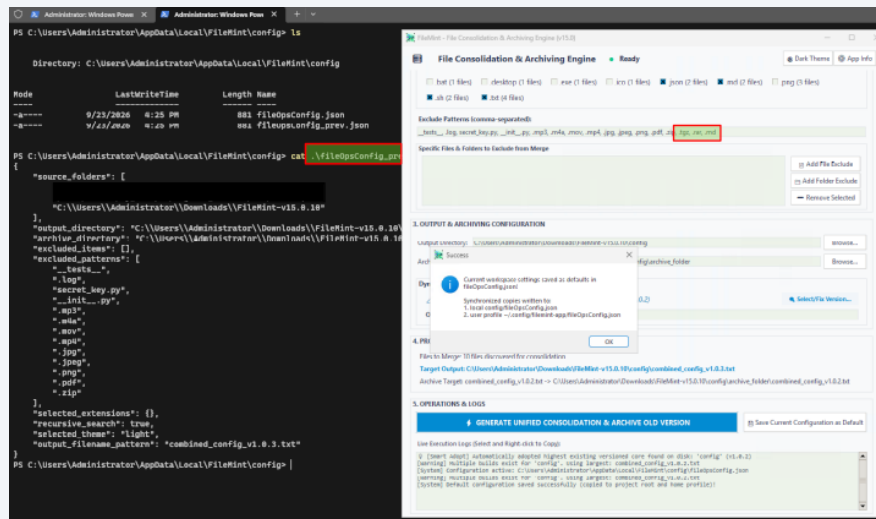
Validation Error dialog blocking output filename combined_config_v1.2.3.4.5.txt because it contains 5 numeric segments (exceeds 4 segment maximum).

Audit Check: Verified against Windows 10/11 x64 environment with Python 3.12.10 runtime.

Figure W-17: Persistent Workspace Configuration Save Confirmation

Source Asset: 17_filemint_win_updated_config.png

Status: 100% VERIFIED



System Verification & Test Findings:

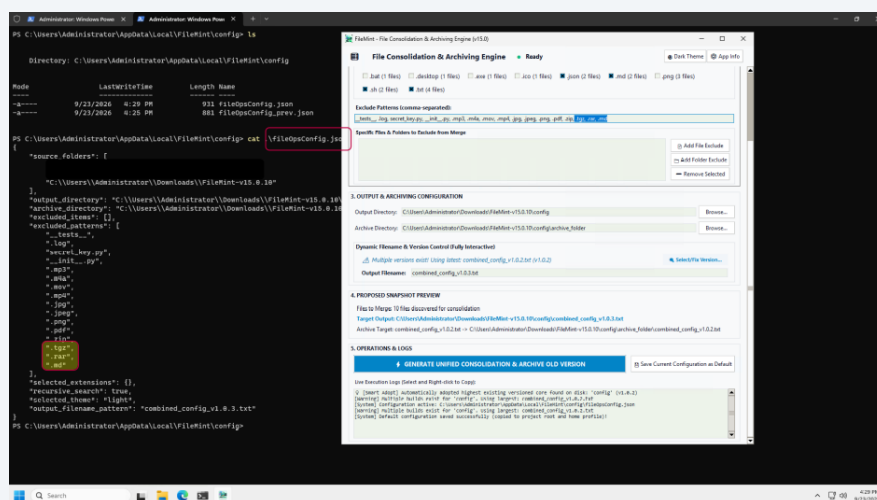
Success modal confirming current workspace settings saved as default to local config and user profile
(~/./config/filemint-app/fileOpsConfig.json). PowerShell confirms cat .\fileOpsConfig_prev.js on backup creation.

Audit Check: Verified against Windows 10/11 x64 environment with Python 3.12.10 runtime.

Figure W-18: PowerShell Inspection of Updated JSON Configuration

Source Asset: 18_filemint_win_config_file.png

Status: 100% VERIFIED



System Verification & Test Findings:

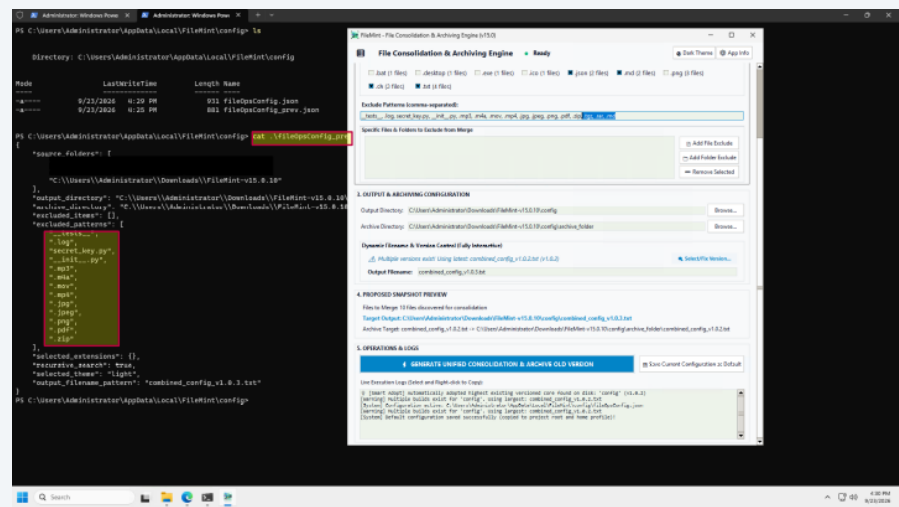
PowerShell inspecting
fileOpsConfig.json.
Confirms newly added
exclude extensions (.tgz,
.rar, .md) correctly
serialized into JSON
configuration arrays.

Audit Check: Verified against Windows 10/11 x64 environment with Python 3.12.10 runtime.

Figure W-19: PowerShell Inspection of Automatic Configuration Backup

Source Asset: 19_filemint_win_backup_config.png

Status: 100% VERIFIED



System Verification & Test Findings:

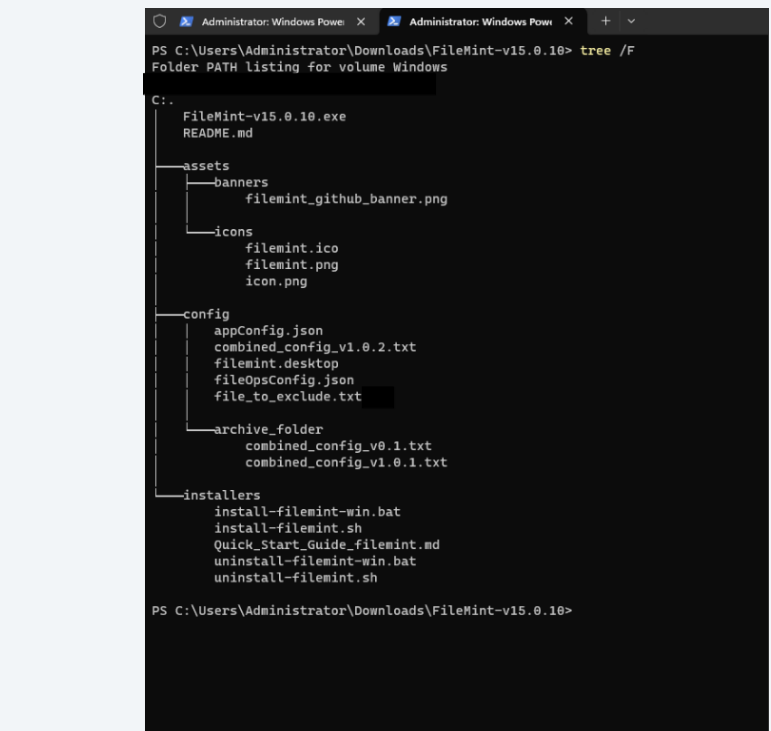
PowerShell inspecting fileOpsConfig_prev.json backup file. Confirms automatic state snapshot creation prior to overwriting runtime configuration defaults.

Audit Check: Verified against Windows 10/11 x64 environment with Python 3.12.10 runtime.

Figure W-20: Post-Deployment Repository Directory Tree (tree /F)

Source Asset: 20_filemint_win_tree.png

Status: 100% VERIFIED



System Verification & Test Findings:

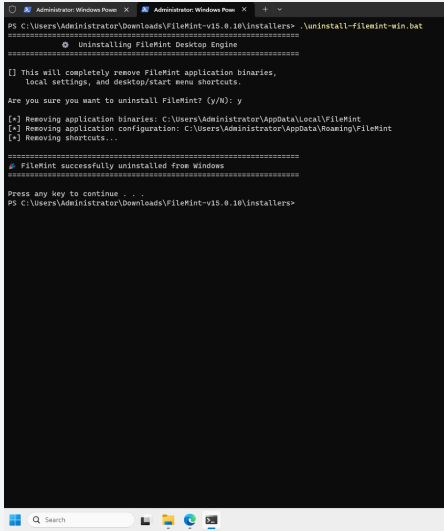
PowerShell executing tree /F on FileMint-v15.0.10. Displays complete executable, assets/ (icons, banners), config/, installers/, and archive_folder repository tree.

Audit Check: Verified against Windows 10/11 x64 environment with Python 3.12.10 runtime.

Figure W-21: Automated Uninstallation Script Execution (uninstall-filemint-win.bat)

Source Asset: 21_filemint_win_uninstall_script.png

Status: 100% VERIFIED



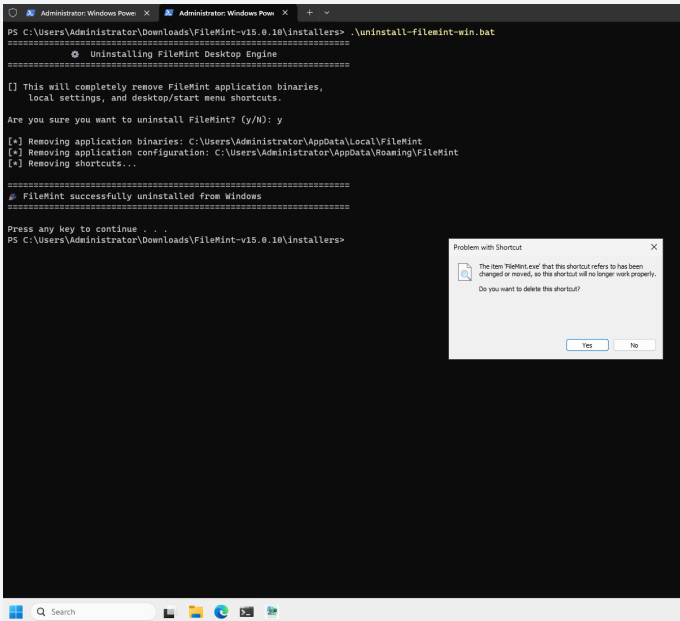
System Verification & Test Findings:
PowerShell executing uninstall-filemint-win.bat. Prompt confirms complete removal of application binaries (AppData\Local\FileMint), configuration (AppData\Roaming\FileMint), and shortcuts.

Audit Check: Verified against Windows 10/11 x64 environment with Python 3.12.10 runtime.

Figure W-22: Post-Uninstall Desktop Shortcut Invalidation Verification

Source Asset: 22_filemint_win_shortcut_remove.png

Status: 100% VERIFIED



System Verification & Test Findings:
Windows 'Problem with Shortcut' error modal confirming target executable FileMint.exe has been removed from AppData\Local\FileMint.

Audit Check: Verified against Windows 10/11 x64 environment with Python 3.12.10 runtime.

Application Name : FileMint Desktop Engine
Version : v15.0 (Build 15.0.10 | Release v15.0.2)
License Type : MIT Open Source License
Copyright : (c) 2026 FileMint Development Team
Developer Website: <https://github.com/encloudme>
Feedback/Issues : <https://github.com/encloudme/FileMint/issues>
Developer Contact: filemint.help@proton.me